

CS 3006

Parallel and Distributed Computing

Lecture 2B

Danyal Farhat

FAST School of Computing

NUCES Lahore

Introduction to Parallel Computing

Lecture's Agenda

- **What Parallel Computing Actually Is?**
- **Motivating Parallelism**
- **Systems vs Computing**
- **Parallel vs Distributed Computing**
- **Scope of Parallel Computing**
- **Why Parallel Programming Is Hard?**
- **Summary**
- **Additional Resources**

What Parallel Computing Actually Is

- A serial (or sequential) program is a recipe carried out by **one worker**, one step at a time.
- A parallel program is the same job reorganized so that **several workers** can be busy at once.

What Parallel Computing Actually Is (Cont.)

Example - One worker, many workers:

- Suppose you must make 8 pizzas, and each pizza takes 4 minutes of dough work, 2 minutes of topping, and 10 minutes in the oven. One chef with **one oven** needs how much time to make 8 pizzas?
- Four chefs with **four ovens** can finish the same job in how much time?

What Parallel Computing Actually Is (Cont.)

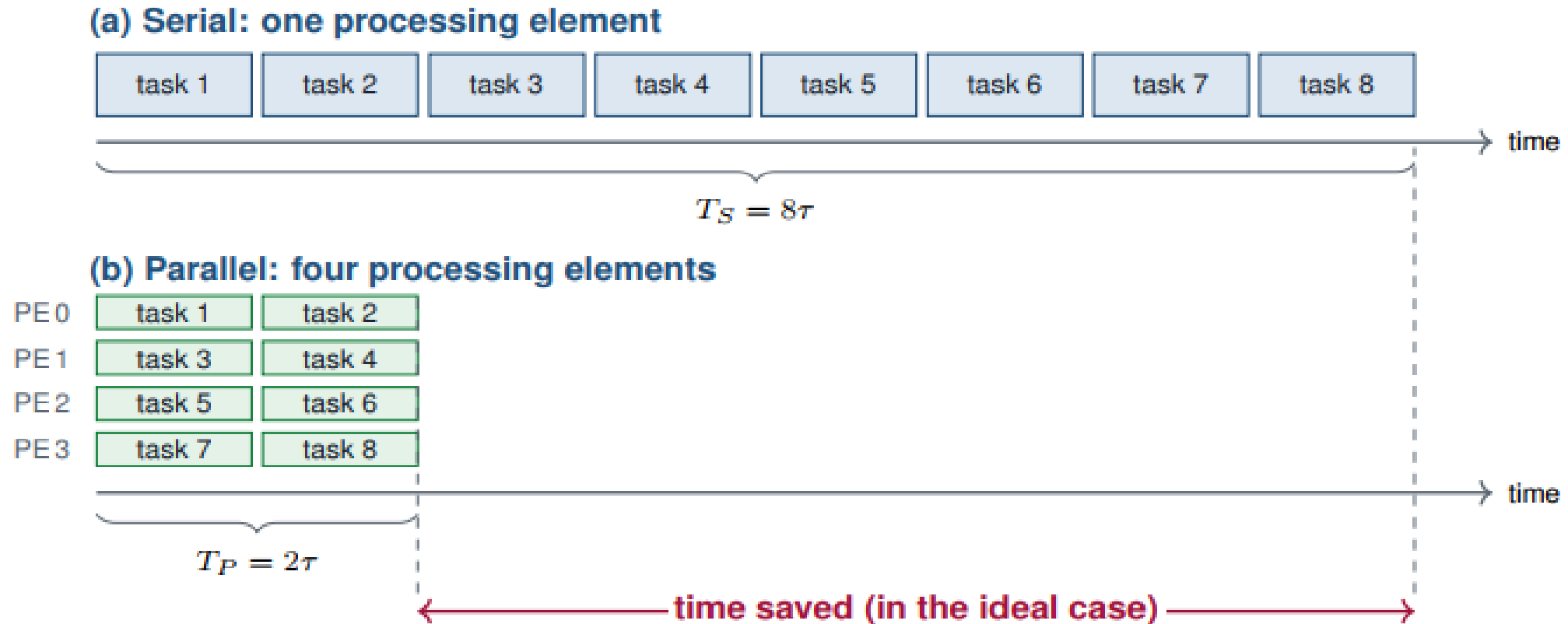


Figure 1: The whole subject in one picture. Eight independent tasks, each taking time τ . One processing element runs them back to back; four processing elements run them two apiece. The ratio T_S/T_P is called the speedup and is formalised in Section 1.4. Everything difficult about parallel computing consists of the reasons the green bars do not really line up that neatly.

What Parallel Computing Actually Is (Cont.)

Concurrency:

- Concurrency is a **property of a program**: two activities are concurrent if their lifetimes overlap and neither has to finish before the other starts.
- A single-core machine running a web browser and a music player is doing concurrency — the operating system interleaves them.

Parallelism:

- Parallelism is a **property of the execution**: two activities are parallel if they are physically happening at the same instant on different hardware.
- Parallelism requires concurrency; concurrency does not require parallelism.

What Parallel Computing Actually Is (Cont.)

Distribution:

- Distribution is parallelism where the workers **do not share memory** and must communicate over a network, so failures of individual workers, and the cost of moving data, become first-class design concerns.
- Concurrency is about **dealing with** lots of things at once; parallelism is about **doing** lots of things at once.

(Rob Pike – Co-creator of Go Language)

What Parallel Computing Actually Is (Cont.)

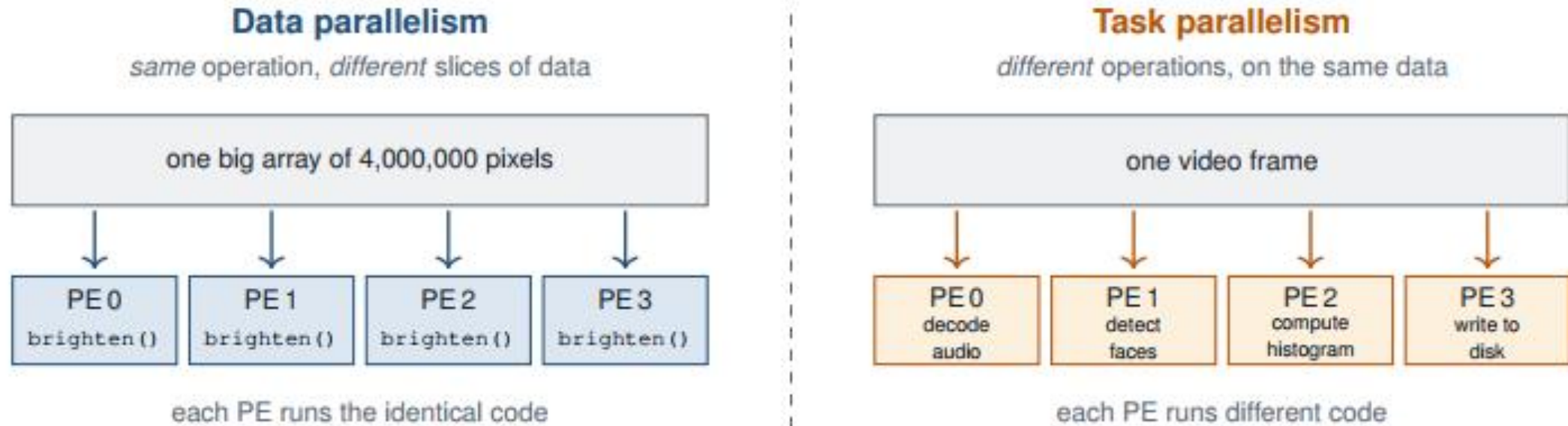


Figure 2: The two decompositions. Data parallelism scales naturally — if you get more pixels, you can use more processing elements. Task parallelism does not: a video pipeline with four stages cannot use a fifth processor no matter how large the video is. This is why most large-scale parallel computing, and essentially all of modern machine learning, is data parallel. Chapter 3 develops both systematically.

What Parallel Computing Actually Is (Cont.)

- Data parallelism is fundamentally a **throughput** play: you are doing more of the same work per unit time, and if the data doubles you simply ask for more processing elements.
- Task parallelism is fundamentally a **latency** play: the work is fixed, and you are trying to get this particular job finished sooner by running its distinct parts at once.
- Pipelining is a **throughput** technique wearing task parallelism's clothes in single-core machine.

What Parallel Computing Actually Is (Cont.)

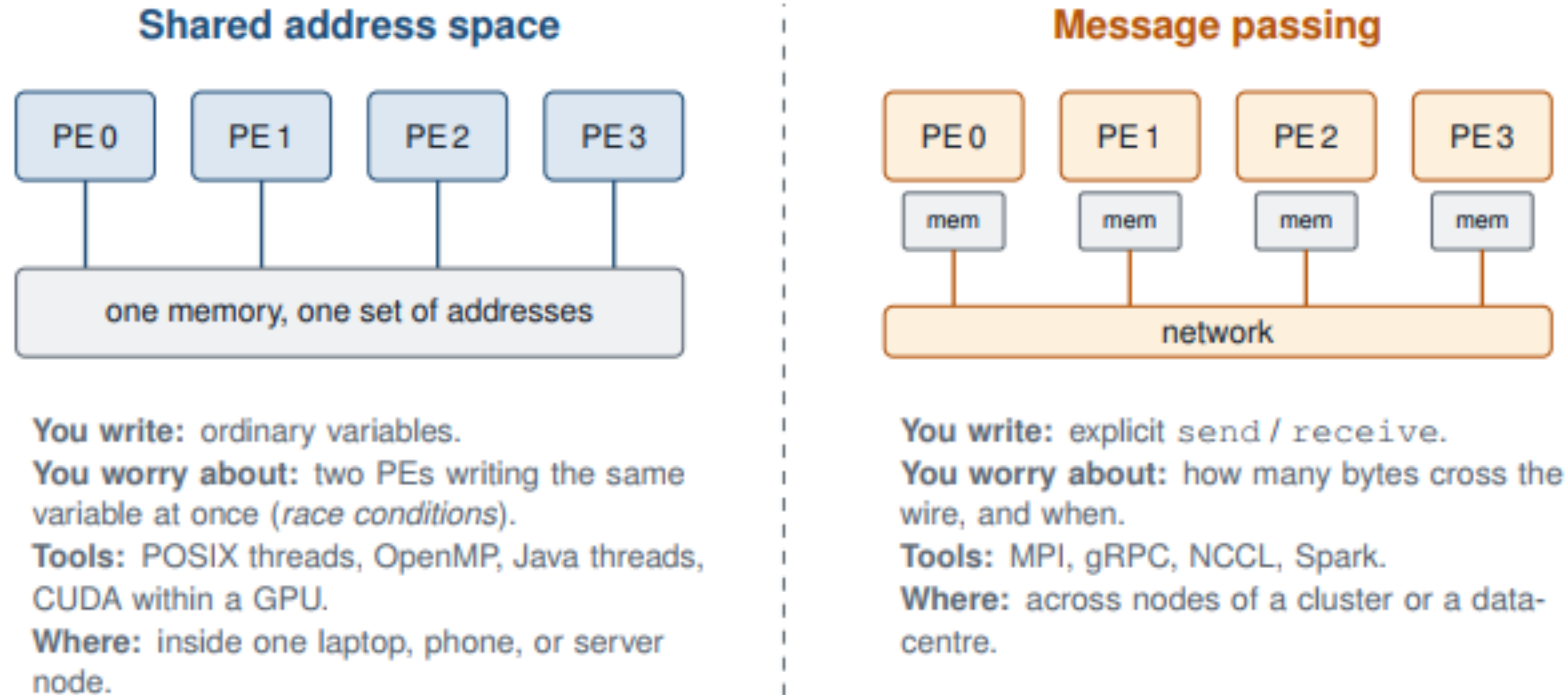


Figure 3: *Shared address space (Chapter 7) versus message passing (Chapter 6). Real 2026 machines are both at once: a GPU cluster uses shared memory inside a node and message passing between nodes. The common idiom “MPI + X” — MPI between nodes, OpenMP or CUDA within a node — exists precisely because the hardware is hierarchical.*

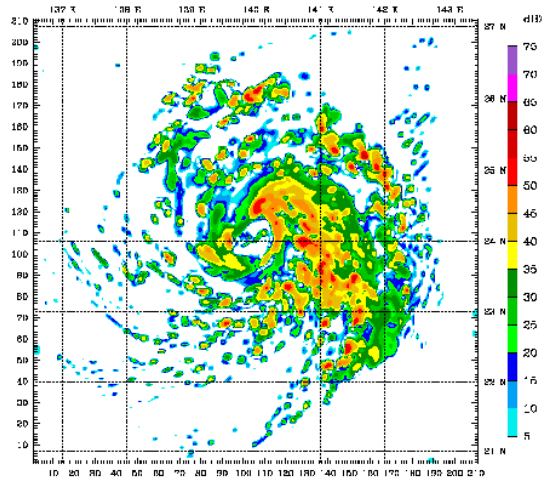
What Parallel Computing Actually Is (Cont.)

Prefix	FLOPS	Something that runs at roughly this rate
mega (M)	10^6	Intel 8087 maths coprocessor, 1980
giga (G)	10^9	a single desktop CPU core in the mid-2000s
tera (T)	10^{12}	ASCI Red, the fastest machine on Earth in 1997; also one modern laptop GPU
peta (P)	10^{15}	IBM Roadrunner, 2008; also a single rack of modern AI accelerators
exa (E)	10^{18}	Frontier, 2022 — the first machine to pass this on the standard benchmark

Table 1: *Each step is a factor of one thousand. Note the compression of the timeline: it took eleven years to get from tera to peta and fourteen to get from peta to exa, but a teraflop machine that filled a room in 1997 is now a component you can buy for a games console.*

Motivating Parallelism

- Uniprocessors are **fast** but
 - Some problems require too much computation
 - Some problems use too much data
 - Some problems have too many parameters to explore
- For example
 - Weather simulations, online gaming, web servers, code breaking



Motivating Parallelism (Cont.)

- Developing **parallel hardware and software** has traditionally been time and effort intensive
- If one is to view this in the context of rapidly improving **uniprocessor speeds**, one is tempted to question the need for parallel computing
- Latest trends in hardware design indicate that uniprocessors are not able to sustain the rate of realizable **performance increments** in the future
- A number of fundamental physical and computational **limitations**
- The **emergence** of standardized parallel programming environments, libraries, and hardware have significantly reduced time to develop (parallel) solution

Motivating Parallelism (Cont.)

2003:

- “uniprocessor . . . architectures may not be able to sustain the rate of realizable performance increments in the future.”

2005:

- Intel cancelled its 4 GHz Pentium 4 and its successors (Tejas and Jayhawk) and pivoted the entire company to **multicore**.
- AMD had already shipped its first **dual-core** Opteron.
- The single-core era ended in 2005.

Motivating Parallelism (Cont.)

2026:

- There is no such thing as a **serial computer** you can buy.
- The question is no longer “**should I parallelise?**” but “how badly am I wasting the 8, 16, or 256 cores I already paid for?”

Motivating Parallelism (Cont.)

The Computational Power Argument – from Transistors to FLOPS

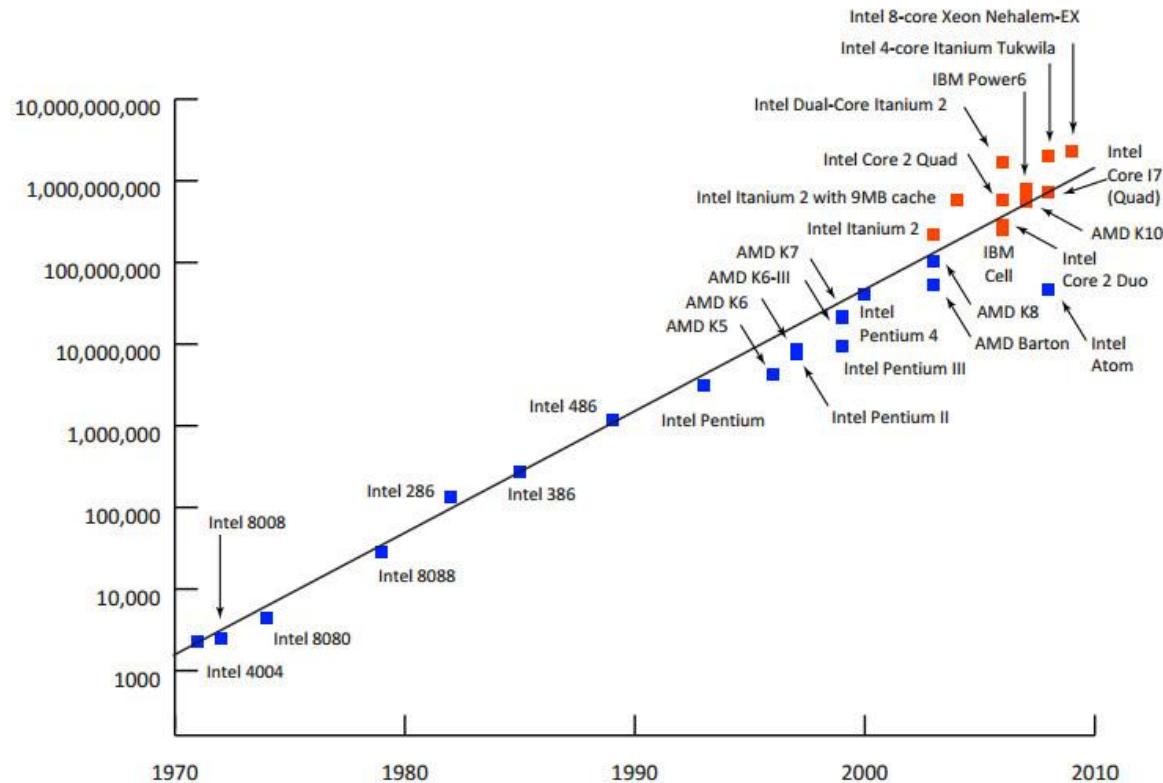
What is Moore's Law?

- Proposed by **Gorden E. Moore** in 1965 and revised in 1975
- It states that [Simplified Version]
 - “Processing speeds, or overall **processing power** for computers will double every 18 months.”
- A more technically correct interpretation
 - “The **number of transistors** on an affordable CPU would double every 18 months.”

Motivating Parallelism (Cont.)

What is Moore's Law? (Cont.)

- Number of transistors incorporated in a chip will approximately **double** every 18 months.



Motivating Parallelism (Cont.)

What is Moore's Law? (Cont.)

- More computational power implicitly **means** more transistors
- Then **why need** second interpretation?
- Let's have a look on **empirical data** from 1970 to 2009
 - In 1970's (i.e., from 1970 to 1979), processor speeds ranged from 740 KHz to 8 Mhz. Difference shows that **both the interpretations** are correct
 - From 2000 to 2009, Speeds ranged from 1.3 GHz to 2.8 GHz
 - ✓ Speed difference is too low but, number of integrated transistors ranged from 37.5 million to 904 million
 - ✓ So, **second interpretation** is more accurate.

Motivating Parallelism (Cont.)

Why doubling the transistors does not doubles the speed?

- The answer is increase in number of transistor per processor is due to **multi-core CPU's**
- In multi-core CPUs, speed will depend on the **proper utilization** of parallel resources
- To follow Moore's law, companies had to:
 - Introduce Ultra Large Scale Integrations (ULSI)
 - ✓ Ultra large-scale integration (ULSI) is the process of embedding millions of transistors on a single silicon semiconductor microchip
 - And multi-core processing era

Motivating Parallelism (Cont.)

Will Moore's law hold forever?

- Adding multiple cores on single chip causes **heat issues**
- Increasing the number of cores, may not be able to increase speeds [Due to **inter-process interactions**]
- Transistors would eventually reach the **limits of miniaturization** at atomic levels

Motivating Parallelism (Cont.)

- So, we must look for **efficient parallel software solutions** to fulfill our future computational needs
- As stated earlier, number of cores on a single chip also have physical and computational **limitations**
- Solution:
 - Need to find more **scalable parallel and hybrid solutions**
- Parallelism is not one option among several for using extra transistors. It is the **only option** that scales.

Motivating Parallelism (Cont.)

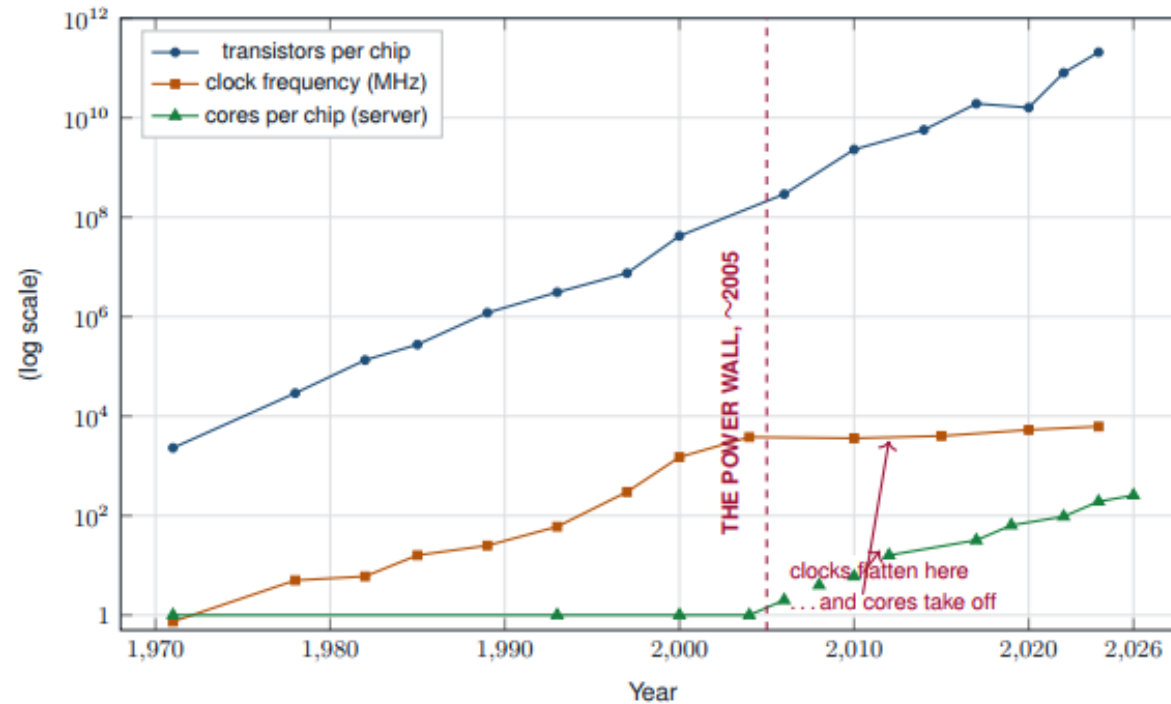


Figure 4: Fifty-five years of microprocessor trends. Transistor count (blue) keeps climbing on essentially the straight line of equation (1). Clock frequency (orange) climbed with it until about 2005 and then stopped dead — the fastest shipping desktop clock has moved less than a factor of two in twenty years, after moving a factor of 5,000 in the thirty before. Core count (green) was pinned at 1 for thirty-three years and has grown by more than two orders of magnitude since. **The transistors did not stop arriving. What stopped was our ability to turn them into a faster single thread.** Every source for these points is listed in Section 1.7.

Motivating Parallelism (Cont.)

The Memory / Disk Speed Argument

- While **clock rates** of high-end processors have increased at roughly 40% per year over the past decade, DRAM access times have only improved at the rate of roughly 10% per year over this interval
- This **mismatch in speeds** causes significant performance bottlenecks
- Parallel platforms provide **increased bandwidth** to the memory system

Motivating Parallelism (Cont.)

The Memory / Disk Speed Argument (Cont.)

- Parallel platforms also provide **higher aggregate caches**
- Some of the fastest growing applications of parallel computing utilize not their raw computational speed, rather their **ability to pump data** to memory and disk faster
 - CPU is fast but memory isn't, which is a **mismatch** in context of parallel programming
 - Solution: Parallel platform (increased **bandwidth** to memory) and combination or collection of caches

Motivating Parallelism (Cont.)

The Memory / Disk Speed Argument (Cont.)

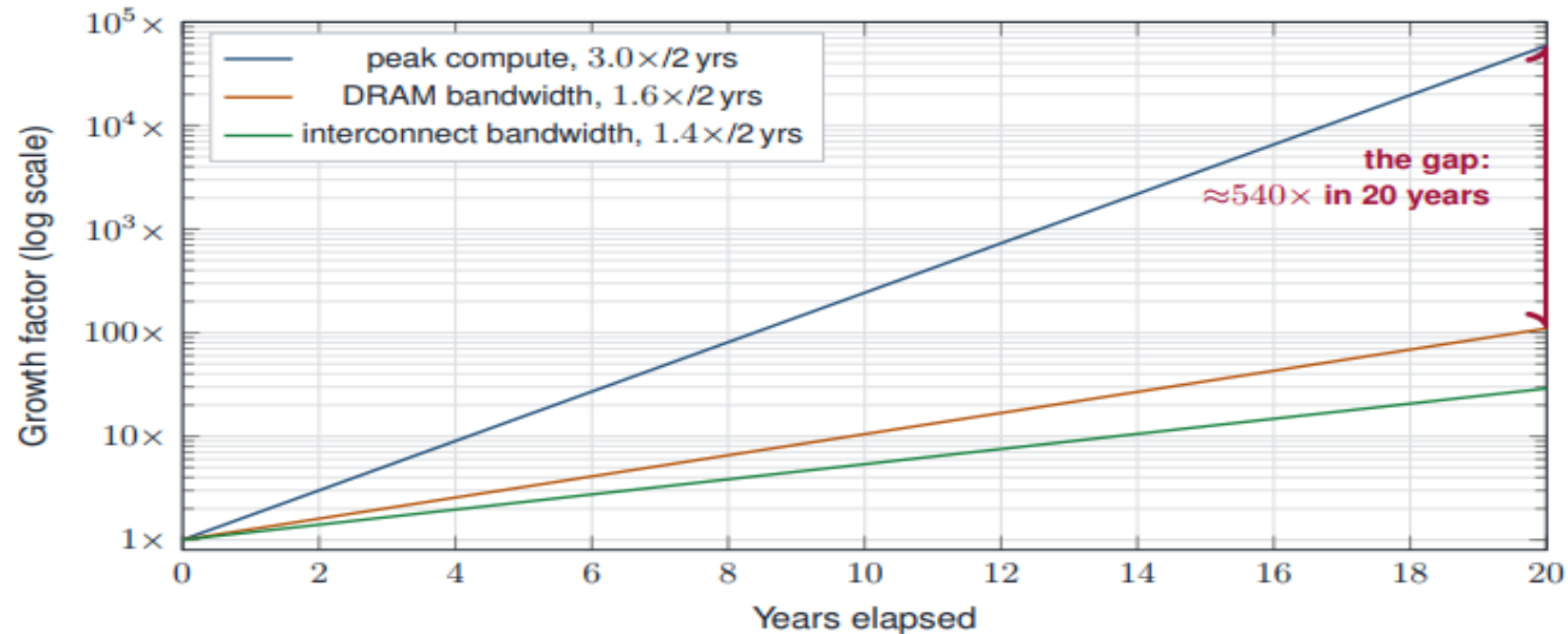
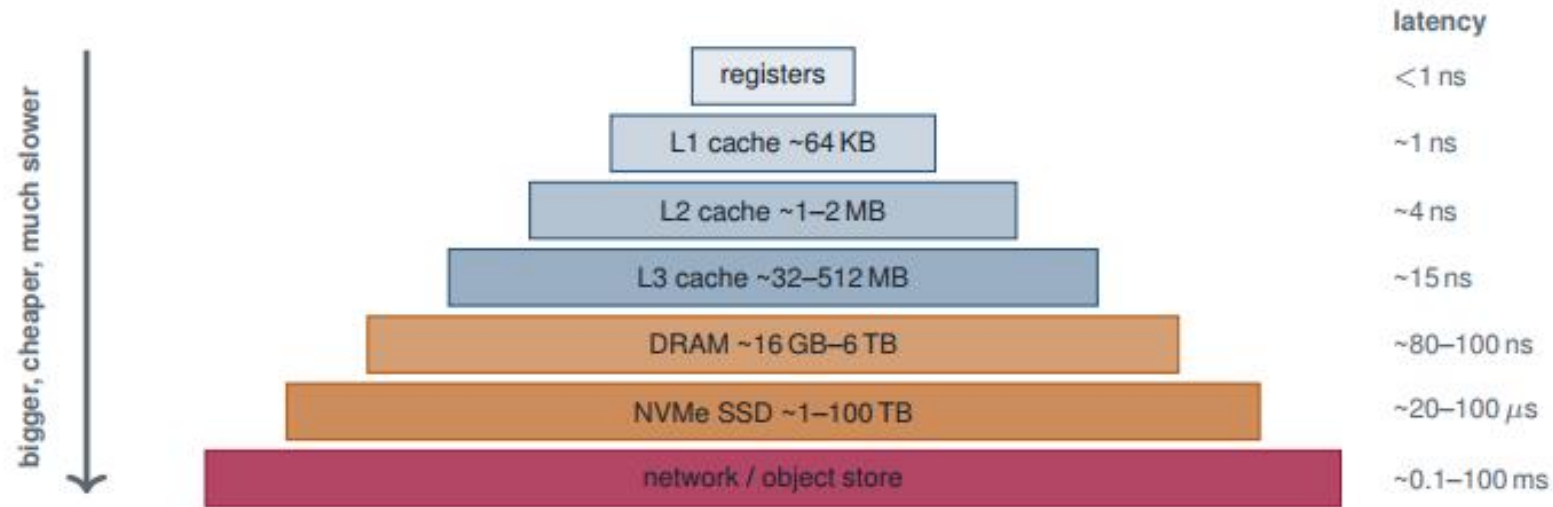


Figure 5: The memory wall, drawn from the growth rates measured by Gholami et al. (2024). All three lines rise — memory really did get better — but because they rise at different exponential rates, the vertical distance between them (which is a ratio, since the axis is logarithmic) grows without bound. A processor that can perform ten thousand times more arithmetic but only fetch a hundred times more data is a processor that spends most of its life waiting.

Motivating Parallelism (Cont.)

The Memory / Disk Speed Argument (Cont.)

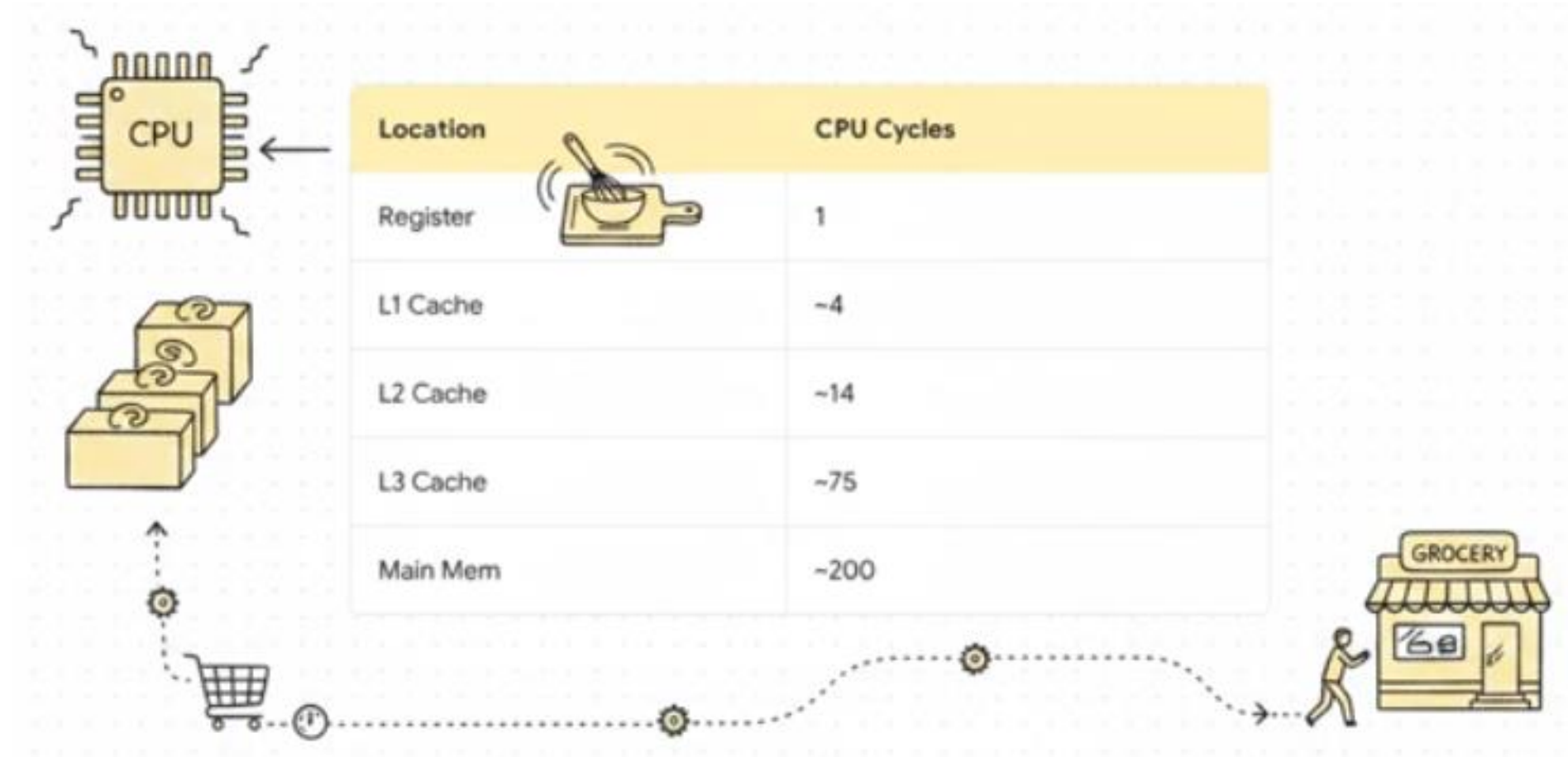


Each step down is roughly an order of magnitude. From an L1 hit to a network round trip is a factor of about **one million**. If your algorithm's data placement is wrong, no amount of parallelism will save you.

Figure 6: The memory hierarchy on a 2026 server, with representative latencies. The 2003 edition's "hierarchy of successively faster memory devices called caches" has since grown more levels, not fewer, and picked up a new one at the bottom: for large-scale computations the "disk" is now often an object store on the other side of a network. The out-of-core techniques the original text mentions have not gone away; they have been renamed and moved up a level.

Motivating Parallelism (Cont.)

The Memory / Disk Speed Argument (Cont.)



Motivating Parallelism (Cont.)

The Data Communication Argument

- As the network evolves, the vision of the Internet as **one large computing platform** has emerged
- In many applications like databases and data mining problems, the **volume of data** is such that they cannot be moved
- Any **analyses** on this data must be performed over the network using parallel techniques

Systems vs Computing

Distributed Systems:

- “A collection of autonomous computers, connected through a network and **distribution middleware**.”
- Enables computers to **coordinate** their activities and to share the resources of the system
- The system is usually perceived as a single, **integrated** computing facility
- Mostly concerned with the **hardware-based** accelerations

Systems vs Computing (Cont.)

Distributed Computing:

- “A specific use of distributed systems to split a large and complex processing into subparts and execute them in **parallel** to increase the productivity.”
- Computing mainly concerned with **software-based** accelerations (i.e., designing and implementing algorithms)

Parallel vs Distributed Computing

Parallel Computing:

- The term is usually used for developing **concurrent solutions** for following two types of the systems:
 - Multi-core Architecture
 - Many core architectures (i.e. CPU's & GPU's)

Distributed Computing:

- The type of computing mainly concerned with **developing algorithms** for the distributed cluster systems
- Here distributed means a **geographical distance** between the computers without any shared-Memory

Parallel vs Distributed Computing (Cont.)

Parallel Computing:

- Shared memory
- Multiprocessors
- Processors share logical address spaces
- Processors share physical memory
- Referred to the study of parallel algorithms

Distributed Computing:

- Distributed memory
- Clusters of workstations
- Processors do not share address space
- There is no sharing of physical memory
- Study of theoretical distributed algorithms or neural networks

Scope of Parallel Computing

Scientific Applications:

- Functional and structural characterization of **genes** and proteins
- Applications in **astrophysics** have explored the evolution of galaxies, thermonuclear processes, and the analysis of extremely large datasets from telescope
- Advances in **computational physics and chemistry** have explored new materials, understanding of chemical pathways, and more efficient processes

Scope of Parallel Computing (Cont.)

Scientific Applications (Cont.):

- Weather modeling for **simulating** the track of natural hazards like extreme cyclones (storms), flood prediction
- Large Hydron Collider (LHC) at European Organization for Nuclear Research (CERN) **generates petabytes of data** for a single collision
 - The data is then distributed to hundred of sites in dozen of countries

Scope of Parallel Computing (Cont.)

Commercial Applications:

- Some of the largest parallel computers power **Wall Street!**
- Data mining analysis for **optimizing business** and marketing decisions
- Large scale servers (**mail and web servers**) are often implemented using parallel and distributed platforms
- Applications such as **information retrieval and search** are typically powered by large clusters

Scope of Parallel Computing (Cont.)

Computer Systems Applications:

- Network intrusion detection: a large amount of data needs to be **analyzed** and processed
- Cryptography (the art of writing or solving codes) employs **parallel infrastructures** and algorithms to solve complex codes – SHA 256
- Graphics processing – Each **pixel** represents a binary
- Embedded systems increasingly **rely** on distributed control algorithms - Modern automobiles use embedded systems

Why Parallel Programming is Hard?

Dependencies:

- If step B needs the result of step A, no amount of hardware makes them simultaneous.
- Finding the **dependencies** is the first job of parallel algorithm design, and the length of the longest chain of them — the critical path — is a hard ceiling on how fast the computation can ever finish.

Why Parallel Programming is Hard? (Cont.)

Communication and Synchronization Cost:

- Splitting work means the pieces **must exchange** results. That exchange takes time, and if the pieces are too small the exchange costs more than the work saved.
- We get these **communication costs** using start-up latency t_s plus a per-word transfer time t_w , and it is the t_s that punishes fine-grained decompositions.

Why Parallel Programming is Hard? (Cont.)

Load Imbalance:

- A parallel computation finishes when its **slowest** processing element finishes.
- If one element gets 40% of the work and the other seven share the rest, seven of them are **idle** at the end.

Why Parallel Programming is Hard? (Cont.)

Non-Determinism:

- A serial program run twice gives the **same** answer.
- A parallel program may **interleave differently** every time, so bugs appear once in ten thousand runs and vanish under a debugger.
- Worse, floating point addition is not associative: $(a + b) + c \neq a + (b + c)$ in general, so even a correct parallel sum can give a **slightly different** answer than the serial one, and a different one on each run if the reduction order varies. This routinely alarms people who assume they have a bug.

Why Parallel Programming is Hard? (Cont.)

Correctness Hazards without Serial Analogues:

- Race conditions, deadlock, livelock, starvation and memory-consistency surprises simply **do not exist** in serial programming.

Misconception: “the compiler will do it for me”

- Automatic parallelisation **works well** for simple, regular loops with statically analyzable array accesses, and it has worked well for those since the 1980s.
- It **does not work** for the cases that matter: irregular data structures, pointer-chasing, anything where the compiler cannot prove that two accesses do not alias.

Limitations of Parallel Computing

- Parallel computing requires designing the **proper communication and synchronization mechanisms** between the processes and the sub-tasks.
- Exploring the **proper parallelism** from a problem is a hectic process.
- The program must have **low coupling and high cohesion** however it is difficult to create such programs.
- Parallel computing needs relatively **more technical skills** to code a parallel program.

Summary

- **What Parallel Computing Actually Is?**

- A serial program is a recipe carried out by one worker, one step at a time.
- A parallel program is the same job reorganized so that several workers can be busy at once.
- Concurrency is a property of a program: two activities are concurrent if their lifetimes overlap and neither has to finish before the other starts.
- Parallelism is a property of the execution: two activities are parallel if they are physically happening at the same instant on different hardware.
- A parallel program is defined by two choices: how the work is split (data parallel / task parallel) and how the workers communicate (shared address space / message passing).

Summary (Cont.)

- **Motivating Parallelism**

- Apps like weather simulations, online gaming, web servers
- Uniprocessors cannot provide required performance due to their computational limitations
- **Moore's Law - Number of transistors double every 18 months**
 - ✓ Doubling the transistors does doubles the speed?
 - True for single core chip but not for multi-core CPUs
 - ✓ Will Moore's law hold forever?
 - Heat issues, synchronization issues, limits of miniaturization of transistors

Summary (Cont.)

- **Motivating Parallelism**

- **The Memory / Disk Speed Argument**

- ✓ Speed mismatch between processor and DRAM can be reduced using parallel platforms and aggregated caches

- **The Data Communication Argument**

- ✓ Analyses on large scale data can be performed over the network using parallel techniques

Summary (Cont.)

- **Distributed System vs Distributed Computing**

- **Distributed System** - Collection of autonomous computers, connected through a network and distribution middleware
- **Distributed Computing** - Specific use of distributed systems to split a large problem into subparts and execute them in parallel

- **Parallel Computing vs Distributed Computing**

- **Parallel Computing** - Developing concurrent solutions
- **Distributed Computing** - Autonomous computers on different locations connected via network

Summary (Cont.)

- **Scope of Parallel Computing**
 - **Scientific Applications** - Characterization of genes and proteins, astrophysics applications, computational physics and chemistry, weather modeling, nuclear physics
 - **Commercial Applications** - Processing of stock exchanges, banking sector, business analytics, web servers, mail servers, search engines
 - **Computer Systems Applications** - Network intrusion detection, cryptography, graphic processing, embedded systems control

Summary (Cont.)

- **Why Parallel Programming is Hard?**
 - **Dependencies:** If step B needs the result of step A, no amount of hardware makes them simultaneous.
 - **Communication and Synchronization Cost:** Splitting work means the pieces must exchange results. That exchange takes time, and if the pieces are too small the exchange costs more than the work saved.
 - **Load Imbalance:** A parallel computation finishes when its slowest processing element finishes.

Summary (Cont.)

- **Why Parallel Programming is Hard?**
 - **Non-Determinism:** A serial program run twice gives the same answer. A parallel program may interleave differently every time, it run.
 - **Correctness Hazards without Serial Analogues:** Race conditions, deadlock, livelock, starvation and memory-consistency are challenges in parallel programming.

Summary (Cont.)

- **Limitations of Parallel Computing**

- **Parallel computing requires designing proper communication and synchronization mechanisms**
- **Parallel program must have low coupling and high cohesion**

Additional Resources

- **Introduction to Parallel Computing** by Ananth Grama and Anshul Gupta

- **Chapter 1: Introduction to Parallel Computing**

- ✓ 1.0 What Parallel Computing Actually Is
- ✓ 1.1 Motivating Parallelism
- ✓ 1.2 Scope of Parallel Computing
- ✓ 1.3 Why Parallel Programming Is Hard

Questions?